MODUL PRAKTIKUM 9

Rancang Bangun Sistem Monitoring dan Prediksi Kepadatan Kendaraan Berbasis Big Data Framework

(Smart City AI Traffic)

Programming Language	: **Python**
Development Environment	: ***Linux Server Environment***
Code Editor	: **VS Code** (*Remote* ***WSL***)
Command Line Interface	: **Bash**
Distributed Data Processing Engine	: **Apache Spark (PySpark)**
Machine Learning Models	: **Linear Regression**
Real-Time Visualization	: **Streamlit**
Durasi	: **150 – 180 menit**

---

1. TUJUAN PRAKTIKUM

Mahasiswa mampu:

- Mahasiswa mampu mengintegrasikan Data Processing Engine (PySpark) dengan Visualization Layer (Streamlit).
- Mahasiswa memahami pentingnya penggunaan format data Parquet untuk efisiensi penyimpanan Big Data.
- Mahasiswa mampu menangani kendala Path Management (Absolute vs Relative) pada Linux Server/WSL.
- Mahasiswa dapat mengimplementasikan model Machine Learning sederhana untuk prediksi deret waktu (time-series).

2. KONTEKS INDUSTRI

Di era Society 5.0, pemerintah kota menggunakan sistem Adaptive Traffic Control System (ATCS). Data dari ribuan CCTV dan sensor IoT dikumpulkan ke dalam Data Lake. Framework Big Data seperti Apache Spark digunakan untuk membersihkan data dari "noise" dan menyimpannya dalam format terkompresi (Parquet). Hasilnya digunakan oleh pembuat kebijakan untuk memprediksi kemacetan di jam sibuk dan mengatur durasi lampu lalu lintas secara otomatis.

3. ARSITEKTUR PIPELINE

Data Generation → Spark Transformation → Parquet Columnar Storage → ML Modeling → Serving (Streamlit)

3.1 Konsep yang Digunakan

- Apache Spark (Batch Processing): Digunakan untuk melakukan agregasi data dalam jumlah besar dengan efisiensi memori yang tinggi melalui sistem lazy evaluation.
- Parquet Storage: Berbeda dengan CSV, Parquet menyimpan data per kolom, memungkinkan Streamlit membaca data ribuan baris dalam hitungan milidetik.
- Linear Regression: Teknik statistik untuk memodelkan hubungan antara waktu (jam) dan volume kendaraan guna mendapatkan nilai prediksi di masa depan.
- Path Absolute: Menentukan lokasi file mulai dari root (/) untuk menjamin stabilitas aplikasi saat dijalankan sebagai layanan server.

4. LANGKAH-LANGKAH PRAKTIKUM

Langkah 1: Setup Environment

Buka terminal Bash (Remote WSL) dan pastikan dependensi Python terpenuhi:

```
pip install pyspark streamlit plotly scikit-learn pandas setuptools
```

Langkah 2: Membangun Engine (main_uts_NIM.py)

Sesi 1: Menyiapkan Alat dan Lokasi Kerja

Di bagian ini, kita memanggil pustaka (library) yang diperlukan dan menentukan alamat folder agar komputer tidak "bingung" saat mencari file di sistem Linux

```
# ====================================================
# MAIN UTS BIG DATA - (STABLE)
# ====================================================

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, when, window, sum as _sum, hour
import random
from datetime import datetime, timedelta
import os
import shutil

# Mendapatkan Absolute Path agar tidak terjadi error folder di Linux
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
OUTPUT_DIR = os.path.join(BASE_DIR, "output")
```

Sesi 2: Menyalakan Mesin Utama (Spark)

Di sini kita menghidupkan Apache Spark, yaitu mesin super cepat untuk mengolah data besar. Kita juga mengatur agar mesin tidak terlalu "berisik" dengan hanya menampilkan pesan jika terjadi error saja.

```
# ============================
# INIT SPARK
# ============================
spark = SparkSession.builder \
    .appName("UTS_BigData_Processing") \
    .config("spark.sql.parquet.compression.codec", "snappy") \
    .getOrCreate()
spark.sparkContext.setLogLevel("ERROR")

print("🚀 Spark Ready - Memulai Pemrosesan...")
```

Sesi 3: Membersihkan Gudang Penyimpanan

Sebelum menyimpan hasil baru, kita harus memastikan tempatnya bersih. Jika folder "output" lama masih ada, kita hapus dan buat yang baru agar data tidak tercampur.

```
# ============================
# PREPARE OUTPUT FOLDER
# ============================
# Membersihkan folder lama agar tidak terjadi konflik izin akses
if os.path.exists(OUTPUT_DIR):
    shutil.rmtree(OUTPUT_DIR)
os.makedirs(OUTPUT_DIR, exist_ok=True)
```

Sesi 4: Membuat Data Simulasi (Data Palsu)

Karena kita sedang mencoba sistem, kita membuat data tiruan yang seolah-olah berasal dari sensor jalan di tiga area berbeda selama 100 menit.

```
# ============================
# GENERATE DUMMY DATA
# ============================
locations = ["AreaA", "AreaB", "AreaC"]
start_time = datetime(2026, 1, 1, 7, 0)

sensor_data = []
for i in range(100):
    for loc in locations:
        sensor_data.append((
            start_time + timedelta(minutes=i),
            loc,
            random.randint(10, 100)
        ))

sensor_df = spark.createDataFrame(sensor_data, ["timestamp", "location", "vehicle_count"])
```

Mengubah daftar tadi menjadi tabel data Spark

sensor_df = spark.createDataFrame(sensor_data, ["timestamp", "location", "vehicle_count"])

Sesi 5: Mengolah Data Menjadi Informasi

Inilah tahap "memasak" data. Kita membuat tiga jenis informasi berbeda:
Total: Menghitung total kendaraan per lokasi.
Tren Waktu: Mengelompokkan data setiap 10 menit.
Data AI: Mengambil angka jam saja agar mudah dipelajari oleh Machine Learning.

```
# ============================
# PROCESSING LOGIC
# ============================
# 1. Hitung total kendaraan tiap area
traffic_df = sensor_df.groupBy("location") \
    .agg(_sum("vehicle_count").alias("total_vehicle"))

# 2. Kelompokkan data per 10 menit (Tren)
traffic_time_df = sensor_df.groupBy(
    window(col("timestamp"), "10 minutes"),
    "location"
).agg(_sum("vehicle_count").alias("total_vehicle"))

# 3. Ambil data jamnya saja untuk keperluan AI
ml_df = sensor_df.withColumn("hour", hour(col("timestamp")))
```

Sesi 6: Menyimpan Hasil dalam Format Parquet

Hasil yang sudah matang disimpan ke dalam folder "output". Kita menggunakan format Parquet karena ukurannya sangat kecil dan sangat cepat dibaca oleh Dashboard Streamlit nantinya.

```
# ============================
# SAVE TO PARQUET (CRITICAL SECTION)
# ============================
def save_data(df, folder_name):
    path = os.path.join(OUTPUT_DIR, folder_name)
    print(f"💾 Menyimpan data ke: {path}...")
    df.write.mode("overwrite").parquet(path)

try:
    save_data(traffic_df, "traffic")
    save_data(traffic_time_df, "traffic_time")
    save_data(ml_df, "ml_data")
    print("\n✅ SEMUA DATA BERHASIL DISIMPAN KE FOLDER OUTPUT")
except Exception as e:
    print(f"\n❌ ERROR SAAT MENULIS DATA: {str(e)}")
```

Sesi 7: Mematikan Mesin

Langkah terakhir yang sangat penting: matikan mesin Spark agar memori (RAM) komputer kembali lega dan bisa digunakan untuk menjalankan dashboard.

```
# Tutup Spark agar resource kembali bersih
spark.stop()
print("🔌 Spark Session Closed. Silakan jalankan Streamlit sekarang.")
```

Langkah 3: Membangun Dashboard (dashboard_NIM.py)

Sesi 1: Menyiapkan Bahan Visual (Import Library)

Tahap awal adalah memanggil "asisten" yang ahli dalam membuat tampilan web, grafik, dan perhitungan statistik.

```
# ====================================================
# DASHBOARD STREAMLIT - (STABLE)
# ====================================================

import streamlit as st
from pyspark.sql import SparkSession
import plotly.express as px
import pandas as pd
from sklearn.linear_model import LinearRegression
import os
```

Sesi 2: Pengaturan Alamat & Judul (Config)

Agar dashboard tidak bingung mencari data hasil masakan "Engine", kita tentukan alamat folder output secara otomatis. Kita juga memberi judul besar pada halaman web kita.

```
# ============================
# DYNAMIC PATH CONFIG
# ============================

# Mencari lokasi folder output secara otomatis
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
OUTPUT_DIR = os.path.join(BASE_DIR, "output")

# Mengatur judul dan lebar tampilan web
st.set_page_config(page_title="Traffic Dashboard", layout="wide")
st.title("🚦 Smart City AI Traffic Dashboard")
```

Sesi 3: Menghubungkan Dashboard ke Mesin Data (Init Spark)

Meskipun ini adalah tampilan web, kita tetap butuh bantuan Spark untuk membaca file Parquet yang berukuran besar. Kita menggunakan @st.cache_resource agar mesin tidak perlu menyala ulang setiap kali kita mengklik tombol di web (jadi lebih cepat).

```
# ============================
# INIT SPARK
# ============================
@st.cache_resource
def get_spark():
    return SparkSession.builder.appName("Dashboard_App").getOrCreate()

spark = get_spark()
```

Sesi 4: Mengambil Hidangan dari Dapur (Data Loading)

Di sesi ini, dashboard mengambil data yang sudah disimpan oleh Engine. Jika datanya belum ada (Engine belum dijalankan), dashboard akan memberikan peringatan kepada pengguna.

```
# ============================
# DATA LOADING WITH ERROR HANDLING
# ============================
def load_parquet(folder_name):
    path = os.path.join(OUTPUT_DIR, folder_name)
    if not os.path.exists(path):
        st.error(f"⚠ Folder data '{folder_name}' tidak ditemukan! Jalankan main script dulu.")
        st.stop()
    return spark.read.parquet(path).toPandas()

# Mengambil 3 jenis data: Total, Tren Waktu, dan Data untuk AI
try:
    pdf = load_parquet("traffic")
    pdf_time = load_parquet("traffic_time")
    pdf_ml = load_parquet("ml_data")
except Exception as e:
    st.error(f"Gagal memuat data: {e}")
    st.stop()
```

Sesi 5: Membuat Menu Interaktif (Sidebar & Filter)

Agar pengguna bisa memilih area mana yang ingin dilihat (Area A, B, atau C), kita membuat menu tarik-turun (dropdown) di samping kiri.

```
# ============================
# SIDEBAR & FILTER
# ============================

# Membuat pilihan lokasi di samping kiri (sidebar)
locations = pdf["location"].unique()
selected_loc = st.sidebar.selectbox("Pilih Lokasi Analisis", locations)

# Menyaring data hanya untuk lokasi yang dipilih user
filtered_pdf = pdf[pdf["location"] == selected_loc]
```

Sesi 6: Menampilkan Kartu Informasi (KPI Metrics)

Kita menampilkan angka-angka penting di bagian atas, seperti total kendaraan secara keseluruhan dan total di lokasi yang sedang dipilih.

```
# ============================
# KPI METRICS
# ============================
st.subheader("📊 Key Performance Indicators")
col1, col2 = st.columns(2)
with col1:
    st.metric("Total Kendaraan (All)", int(pdf["total_vehicle"].sum()))
with col2:
    st.metric(f"Total di {selected_loc}", int(filtered_pdf["total_vehicle"].sum()))
```

Sesi 7: Menampilkan Grafik Tren (Visualization)

Sesi ini mengubah angka-angka membosankan menjadi grafik garis yang cantik sehingga kita bisa melihat kapan lalu lintas mulai ramai.

```
# ============================
# VISUALIZATION
# ============================
st.markdown("---")
c1, c2 = st.columns(2)

with c1:
    st.subheader("Traffic Time Series")
    # Memperbaiki format waktu agar bisa dibaca grafik
    pdf_time["start_time"] = pdf_time["window"].apply(lambda x: x[0] if isinstance(x, tuple) else x.start)

    # Membuat grafik garis menggunakan Plotly
    fig_line = px.line(pdf_time, x="start_time", y="total_vehicle", color="location")
    st.plotly_chart(fig_line, use_container_width=True)
```

Sesi 8: Menampilkan Prediksi Pintar (AI Prediction)

Inilah bagian tercanggih. Dashboard menggunakan model Machine Learning untuk menebak jumlah kendaraan di jam tertentu berdasarkan pilihan pengguna.

```
with c2:
    st.subheader("🤖 AI Prediction (Linear Regression)")

    # Menyiapkan data untuk dipelajari AI
    X = pdf_ml[["hour"]]
    y = pdf_ml["vehicle_count"]

    # Melatih AI secara instan
    model = LinearRegression()
    model.fit(X, y)

    # Membuat slider agar user bisa pilih jam (misal jam 5 sore)
    hour_input = st.slider("Prediksi Jam Ke-", 0, 23, 12)
    pred = model.predict([[hour_input]])
    st.success(f"Prediksi jumlah kendaraan pada jam {hour_input}:00 adalah **{int(pred[0])}**")
```

Langkah 4: Eksekusi

1. Jalankan Engine: python main_uts_NIM.py
2. Verifikasi folder: ls output/traffic (Pastikan ada file .parquet)
3. Jalankan Dashboard: streamlit run dashboard_NIM.py

5. CHECKLIST VALIDASI PRAKTIKUM

No	Kriteria Keberhasilan	Check
1	Terminal menampilkan pesan "✅ SEMUA DATA BERHASIL DISIMPAN"	[ ]
2	Terbentuk folder 'output' dengan 3 sub-folder (traffic, traffic_time, ml_data)	[ ]
3	Dashboard Streamlit terbuka di Browser (Port 8501)	[ ]
4	Slider Prediksi AI berfungsi dan mengubah nilai metrik	[ ]
5	Grafik Plotly menampilkan data dari ketiga area (AreaA, AreaB, AreaC)	[ ]

6. OUTPUT WAJIB YANG DIKUMPULKAN

a) Script: main_uts_NIM.py & dashboard_NIM.py
b) Bukti Eksekusi: Screenshot terminal Linux saat penulisan Parquet sukses.
c) Visualisasi: Screenshot Dashboard Streamlit utuh (Header sampai Grafik Prediksi).
d) Dokumen: Laporan singkat analisis jam tersibuk berdasarkan dashboard.

7. RUBRIK PENILAIAN (SKOR 1-4)

Aspek	Skor 1	Skor 2	Skor 3	Skor 4
Data Pipeline	Gagal tulis file	File tulis tapi path salah	Berhasil tulis data	Optimal (Clean & Fast)
Integrasi Parquet	Tidak menggu	Manual/Corrupt	Berhasil terbaca	Implementasi Absolute Path
UI Dashboard	Error/Blank	Minimalis/Statis	Interaktif & Grafik Lengkap	UI Profesional + Plotly
Prediksi AI	Tidak ada	Error library	Muncul angka	Visualisasi Tren Prediksi

8. INSIGHT

Keberhasilan sistem monitoring Big Data bukan hanya pada kecanggihan algoritmanya, melainkan pada ketahanan **Pipeline Data**-nya. Dengan menggunakan format **Parquet** dan **Absolute Path**, kita memastikan data tetap konsisten dan cepat diakses meskipun aplikasi dipindahkan ke lingkungan server yang berbeda (Portability).